WHITEPAPER · DRAFT V0.1 · 2026-08-05

Velum: Holder-Attested Compliance for Confidential Tokens on Stellar

Stellar Summit SP 2026 · Enterprise, Compliance & RWA (OpenZeppelin + Nethermind)

Written in English: the intended readers are the lane's sponsors and the upstream maintainers we hope to contribute back to. A Portuguese edition can follow.

Abstract

Confidential Tokens give Stellar hidden balances with a designated auditor — confidentiality without anonymity. That is the right shape for regulated finance, but it leaves regulated issuers with only half a compliance story. The token's authorization interface receives an address and nothing else, so it can express *who may hold* and never *how much they may hold*: balances are Pedersen commitments, and the interface has no opening for them.

We show this is not an oversight but a structural consequence of commitment-based confidentiality, and that it has a specific remedy. Under fully homomorphic encryption a contract can evaluate a predicate over ciphertext with no witness at all; under Pedersen commitments with zero-knowledge proofs, someone must know the opening — and for any given balance exactly one party does: its holder. Quantitative compliance therefore cannot be *transaction-gated*; it must be **holder-attested**.

Velum implements that conclusion. It contributes (i) `velum-policy`, an adapter making the Confidential Token's authorization gate consult an ERC-3643/T-REX identity registry rather than a flat address list; (ii) `disclose_balance_ge`, an implementation of the predicate disclosure circuit that OpenZeppelin's own selective-disclosure specification defines (§9) and no public implementation provides; and (iii) `velum-attest`, an on-chain verifier for that proof — a component the same specification marks out of scope (§§5.4, 14). A holder proves "*my confidential position is at least T*", a Soroban contract checks it, and no party — including the contract — learns the position. Proving takes 0.25 s and adds 3% to a transfer circuit's constraint count when the same technique is applied there.

1 The problem

An issuer tokenizing a regulated fund on a public ledger faces an uncomfortable trade. The ledger's transparency, normally a feature, publishes the register of holders: who subscribed, how much, and when they left. In retail this never bit — a small position hides in the crowd and attracts no professional adversary. Institutionally the arithmetic inverts. A pool of ten positions is de-anonymized by correlating subscription timing with off-chain events, and the information has a paying audience: competitors, counterparties, trading desks.

Confidential Tokens address exactly this. Balances and transfer amounts become Pedersen commitments on the Grumpkin curve; every state transition carries an UltraHonk zero-knowledge proof verified on-chain; addresses stay public and a designated auditor holds a decryption key. Deposits and withdrawals remain in cleartext, which for a fund maps neatly onto subscription and redemption being auditable while the internal register stays private.

What remains missing is compliance with teeth.

2 Two halves of compliance

Regulation asks two questions of a fund's register, and they have different shapes.

Qualitative — who may hold? The token's compliance extension consults an external policy:

```
trait Policy {
    fn is_authorized(e: Env, account: Address, token: Address) -> bool;
}
```

The reference implementations answer from a flat allowlist or blocklist. That is insufficient for a securities regime: an address on a list carries no reason, no expiry, and no attesting party. Brazil's CVM 175, like ERC-3643's T-REX model, asks whether the holder carries a *valid claim* — qualified investor, say — issued by an *approved* claim issuer. The RWA module in the same library answers precisely that question, through `IdentityVerifier::verify_identity`. The two modules do not know about each other. Bridging them is upstream issue #766, open since 2026-07-06 with no release at the time of writing.

Quantitative — how much may they hold? Minimum ticket, per-investor cap, concentration limit, subordination ratio between tranches. Every one of these is a statement about a *value*, and this is where the interface runs out: `is_authorized` receives an address. It sees no amount and no balance, because there is no amount and no balance to see — only commitments.

Both halves of this section are sourced to upstream's code and specifications in **Appendix A** — including the trait signatures, the sentence stating that the token's only agreement with a policy is a boolean, and the open issue that names the missing bridge.

3 Why the enforcement point must move

It is tempting to conclude that the fix is a richer policy interface. It is not. Consider where the enforcing party would have to obtain the value.

The confidential token's lifecycle hooks reveal the boundary precisely:

HOOK	SEES THE AMOUNT?
`on_deposit(from, to, amount: i128)`	yes — deposits are public
`on_withdraw(from, to, amount: i128, …)`	yes — withdrawals are public
`on_transfer(from, to, payload)`	**no** — the amount is a commitment

Those two signatures are reproduced verbatim in Appendix A.1, alongside the RWA compliance hooks that expect cleartext amounts and balances — the two worlds cannot meet, and the

reason is a type signature.

So amount rules *can* be enforced where value crosses the wrapper's boundary. But in the realistic fund topology the investor never deposits: the distributor holds the public balance and credits investors by confidential transfer, precisely so that individual positions are not derivable from public deposits. The subscription amount is then inside a confidential transfer, and invisible again.

Could the sender prove the recipient's resulting balance respects a cap? No. The recipient's balance is a commitment `C = v·G + r·H` whose opening `(v, r)` the sender does not possess, and Pedersen binding guarantees it cannot find another. The auditor could decrypt — but that makes every transfer wait on an off-chain party, and turns a compliance check into a custody dependency.

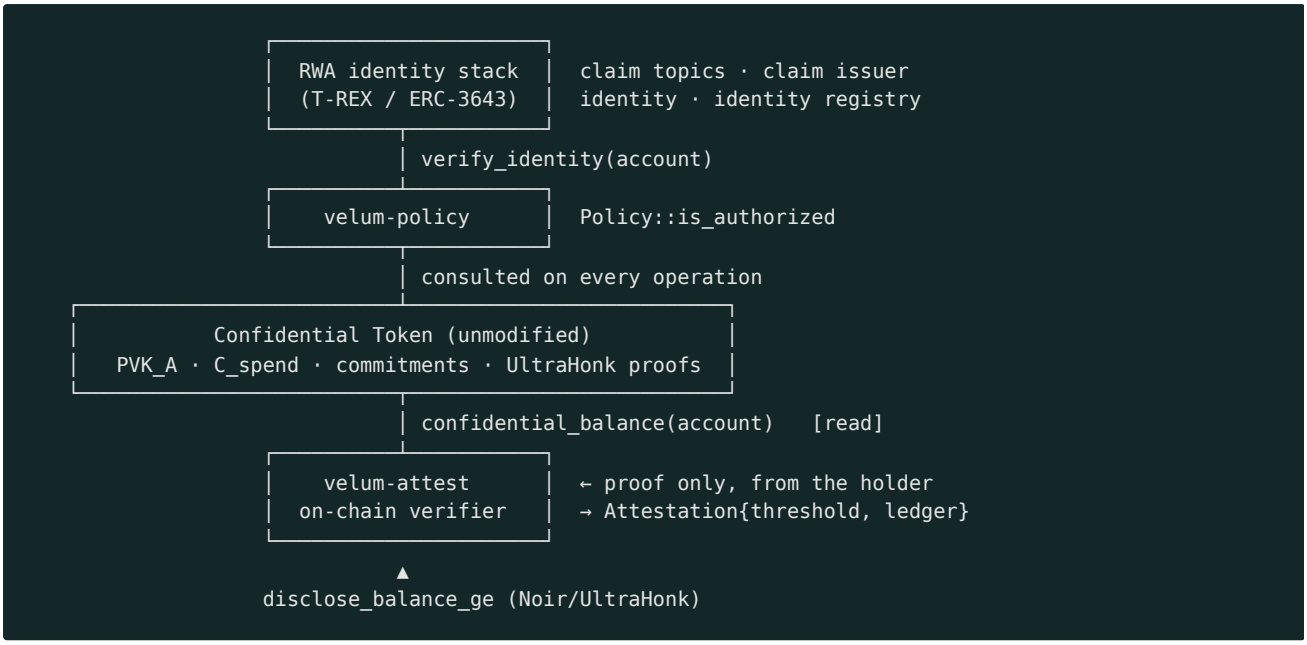
This is where the cryptographic substrate matters. Under FHE — the route taken on the EVM by ERC-7984's balance-cap hook module — a contract compares ciphertexts directly and needs no witness, so the check can sit inside the transfer. Under commitments plus zero-knowledge, a proof requires a witness, and the witness is the opening. For any balance, exactly one party holds it: the account holder, who retains `(v_s, r_s)` as ordinary wallet state.

> **Thesis.** Under commitment-based confidentiality, quantitative compliance cannot be transaction-gated. It must be **holder-attested**: the party that can prove is the party whose balance it is.

This is not a weaker guarantee, only a differently-shaped one. It relocates the check from "every transfer is screened" to "a holder demonstrates standing, and the demonstration is a fact on the ledger". For thresholds that qualify a holder — the qualified-investor minimum, a concentration ceiling — that is exactly the semantics the regulation wants.

4 Architecture

Velum is three components over the unmodified upstream token.



4.1 · velum-policy — identity, not addresses

An adapter, and deliberately small. It resolves an impedance mismatch: the RWA verifier signals failure by panicking, the token's policy expects a boolean, and a panic crossing a cross-contract call would abort the transaction instead of letting the token reject the operation cleanly. The call therefore goes through the generated client's `try_` variant, folding the error into `false`. The claim logic stays in the RWA registry — shared with the public asset — and the confidentiality logic stays in the token. One KYC registry governs both.

4.2 · disclose_balance_ge — the predicate circuit

Upstream's `SELECTIVE_DISCLOSURE.md` §9 specifies a D-balance circuit in two shapes: a value-revealing form that seals the balance to a designated recipient, and a **predicate-only** form (`disclose_balance_ge` / `disclose_balance_le`) in which the proof's mere validity asserts the inequality and there is nothing to decrypt. The specification is complete — public inputs, private witnesses, constraints D1/D2/DB3/DB4/D5. The distributed circuits implement `disclose_recipient` and `disclose_sender` only.

We implement the `ge` predicate to that specification:

#	CONSTRAINT	PURPOSE
D1	`vk_A = Poseidon2(ϑ_vk, sk_A, addr_f)`	contract-bound viewing key
D2	`PVK_A = vk_A · H`	binds the proof to the on-chain account record
DB3	`C_spend = v_s·G + r_s·H`	opening of the current balance; by Pedersen binding the witnessed `v_s` *is* the balance
D5	`v_s ∈ [0, 2¹²⁷)`	prevents a wrapped-negative `v_s` satisfying DB4
DB4	`v_s ≥ v_threshold`	the predicate, as a non-negative difference

We add one constraint beyond the specification:

DB4b	`v_threshold ∈ [0, 2¹²⁷)`	a threshold near the field modulus would make DB4 vacuous by wrap-around

`v_threshold` is contract-supplied rather than prover-supplied, so DB4b is defence in depth rather than a soundness necessity — but it makes the circuit's guarantee independent of how carefully a deployment populates its profile, at negligible cost.

4.3 · velum-attest — verification on-chain

Upstream verifies disclosure proofs entirely off-chain (§15.1) and marks an on-chain verifier out of scope, noting that the predicate shape is what such a verifier would consume (§14). The soundness condition is the specification's own trust boundary (§5.2): **every public input must originate somewhere other than the prover.**

`velum-attest` satisfies it by construction:

PUBLIC INPUT	SOURCE
`PVK_A`, `C_spend`	read cross-contract from the confidential token, live
`addr_f`	pinned at construction (see §7.1)
`v_threshold`	the contract's own regulatory profile, owner-managed
proof	the only value the caller supplies

A caller substituting its own commitment merely verifies against a blob the contract never assembles. On success the contract records `Attestation { threshold, attested_at_ledger }` and emits an event.

The entry point requires no authorization. A proof is self-authenticating: D1/D2 bind it to the account's own viewing key, so no one can produce one for an account they do not control, and relaying a valid proof only records a fact that account could have recorded itself.

5 Freshness and replay

`C_spend` is read from live state, so a proof stops verifying the moment the balance moves. This gives replay resistance without a nonce: an old proof can only be presented against the state it was made for, where it remains true.

The consequence is that an attestation is a statement about the position *as of* its ledger. Consumers needing a current answer re-attest; consumers accepting a window read `attested_at_ledger` and decide. Raising the threshold does not silently upgrade old records — each attestation retains the threshold it was checked against.

6 What this does not solve

Stated plainly, because a compliance claim that quietly enforces nothing is worse than no claim.

Aggregates. Concentration ("no holder above X% of the fund") and the subordination ratio between senior and subordinated tranches are statements over *sums across accounts*. Pedersen commitments are additively homomorphic, so the sums are computable on-chain — but opening

them requires a party that knows every contributing opening, which is the auditor. Concentration against a *public* total is self-provable by the holder and fits this construction directly; a ratio between two encrypted tranche totals does not. We consider this the most interesting open problem in the area.

Third-party balance caps. A cap on a *recipient's* balance at transfer time remains unprovable for the reasons in §3. Enforcement at consolidation (`merge`) is the natural alternative — the holder proves the merged balance respects the cap — but `merge` carries no proof upstream, so this requires a contract-level fork we did not attempt.

Involuntary movement. Everything above is holder-attested, which by construction excludes the case where the holder will not cooperate: individual clawback. That is upstream's own open problem, and §11 shows it reduces to a question with a short answer.

The accept/reject side channel. A verified attestation reveals one bit: the position is at or above the threshold. Repeated attestations against a moving threshold would binary-search the balance. Deployments should treat the threshold as policy, not as a query interface. Upstream documents the analogous leak in its own hook design.

Audit status. The UltraHonk backend and the confidential circuits are unaudited developer preview. Testnet only. Our own circuit inherits that status and adds itself to the surface.

7 Findings for upstream

Three defects and gaps found while building; we intend to report all three.

7.1 · `addr_f` is not readable by third parties. The address-as-field element lives in the token's instance storage and `address_to_field` is `pub(crate)`, so an external verifier can neither read nor recompute it. §5.3 of the disclosure specification lists it as "recomputed from the token contract address", which no third-party contract can do. `velum-attest` pins it at construction — still not prover-supplied, so the trust boundary holds — but a public `address_as_field()` accessor on `ConfidentialToken` would let it be read live like every other public input.

7.2 · A documented command that cannot run. `examples/rwa/sign-claim` is listed in the workspace `exclude` set while declaring `authors.workspace = true`. The README's `cargo run --manifest-path examples/rwa/sign-claim/Cargo.toml` therefore fails with "*failed to find a workspace root*".

7.3 · `--optimize=false` is incompatible with stellar-cli ≥ 25.2, which treats `--optimize` as a boolean flag; the demo's deploy script fails at the first contract.

8 Evaluation

Measured on commodity hardware (12 threads), `nargo 1.0.0-beta.11`, `bb 0.87.0`.

QUANTITY	VALUE
`disclose_balance_ge` circuit tests	10 / 10 pass
Witness generation	0.07 s
Verification-key size	1 764 B
Proof generation	**0.25 s**
Proof size	14 592 B
Public inputs	6 fields (192 B)
Verification	succeeds; a false claim is not even witnessable

The test suite covers acceptance above, at, and with a zero threshold; and rejection for a balance below threshold, a false commitment opening, another holder's secret key, replay against a different token (`addr_f`), a stale commitment, and a malformed threshold near the field modulus.

As a separate datapoint on cost, applying the same technique inside the *transfer* circuit — a per-operation cap and a minimum ticket, both over the encrypted amount — raised its constraint count from 133 to 137 ACIR opcodes (**+3%**) with all 28 upstream tests still passing. Range comparison is already the circuit family's dominant idiom; regulatory predicates are cheap guests.

For context on the surrounding stack: building the seven upstream contracts takes 57 s, deploying them to testnet 1 m 56 s, and a full confidential flow (register → deposit → merge → transfer → merge → withdraw, every proof verified on-chain) 1 m 4 s.

9 Related work

FHE on the EVM. OpenZeppelin's Confidential Contracts v0.5 ships `ERC7984BalanceCapHookModule` and `ERC7984HolderCapHookModule`, audited June 2026, alongside an `ERC7984IdentityCheck` consuming an ERC-3643-style registry. Fully homomorphic encryption lets the pre-transfer hook compare ciphertexts directly and zero the amount on failure, with no witness and no prover. This is the closest prior art and the cleanest contrast: it does not face the prover problem, and correspondingly needs a different execution substrate. Zama has announced a confidentiality layer for the T-REX Ledger; the chain's mainnet is scheduled after this writing.

Auditor keys without value rules. Avalanche's eERC (zk-SNARKs with partially homomorphic encryption, rotatable auditor keys) and Solana's Token-2022 Confidential Balances (ElGamal with equality, range and validity proofs, optional auditor key) both deliver confidentiality with audit access, and neither enforces quantitative limits over encrypted balances.

Permissioned ledgers. Canton and bank-operated networks resolve privacy and compliance together by restricting who sees the ledger at all, at the cost of permissionless composability and shared liquidity.

Primitives. Range proofs and proof-of-solvency are long-settled cryptography. What has been missing is not the primitive but the composition: a regulatory predicate over a confidential

balance, verified by a contract, wired to an identity registry.

10 Regulatory mapping

The anchor case is a Brazilian receivables fund (FIDC) under CVM 175, tokenized as senior and subordinated classes. The mapping is deliberately narrow:

RULE	MECHANISM	STATUS
Holder must be a qualified investor	claim topic in the identity registry, enforced on every operation via `velum-policy`	implemented
Position must clear the qualified-investor minimum	`disclose_balance_ge` + `velum-attest`	implemented
Position must stay under a concentration ceiling (public total)	`disclose_balance_le`, same construction	specified, not built
Subordination ratio between classes	encrypted aggregate across accounts	open problem (§6)
Issuer freeze, sanction response	upstream freeze and SAC passthrough	inherited

A jurisdiction is expressed as a profile — claim topics, thresholds, modules — so the machine is reusable and the regulation is the configuration.

11 Design note: individual clawback under confidentiality

We did not implement the migration this requires; §11.6 prices it. Everything short of it, however, is **implemented and proven**: the cryptographic premise is verified against the unmodified upstream circuit library (`experiments/clawback-poc`, 9/9 tests), and the seize circuit itself is built, tested and **verified on-chain** (`circuits/seize` — 12/12 tests, 93 ACIR opcodes; `velum-seize` on testnet verified a seizure bounded by a live confidential position without the position being readable, and an `alpha` above the balance is not even witnessable). The note is included because it answers a question upstream left open, and because the answer turns out to be short.

11.1 · The problem

Once value enters the wrapper, the underlying SEP-41 ledger records the *contract* as holder. An issuer's SAC-level clawback would drain the pool, debiting unrelated accounts. Individual seizure must instead extract value from one confidential account and settle it over a transparent path — and the contract does not know that account's balance.

Upstream specifies such a flow (`COMPLIANCE.md` §5): the admin authorizes, the auditor supplies knowledge, and a circuit bounds the seized amount by the target's balance. The specification then defers its own crux, the treatment of the spendable-balance blinding `r_s`:

> "The follow-up revision will pin down whether `r_s` is supplied as a private witness with an auxiliary opening proof or derived in-circuit from a separately escrowed value."

The passage is quoted at greater length in Appendix A.4, where upstream also states the obstacle outright — "*because the clawback circuit does not have access to* `vk_A`" — and lists "derived in-circuit from a separately escrowed value" as one of the two candidate remedies. This note takes that option and names the value.

The difficulty is real. The auditor recovers the *value* `v_s` from the sender-channel checkpoint, and the full opening `(v_r, r_r)` of the receiving side by replaying per-transfer ciphertexts. It cannot recover `r_s`, which derives from the holder's viewing key.

11.2 · The observation

`r_s` is not sampled. The shared circuit library states the rule the protocol uses in every owner-initiated operation (constraints W5 / T10 / S9 / V6):

```
derive_spend_r(vk, sigma) = Poseidon2(SPEND_RANDOMNESS, [vk, sigma])
```

It is a deterministic function of the holder's viewing key and a nonce `sigma` that is **public** — it travels in the event. The auditor has `sigma`. The only missing input is `vk_A`.

The open question therefore reduces to one: *can the auditor obtain* `vk_A`*?*

The reduction goes further than the specification suggests. The balance checkpoint emitted on every owner-initiated operation is `b~ = v + Poseidon2(ENCRYPTED_BALANCE, [vk, sigma])` — also decryptable with `vk` alone. So `vk` recovers the **value and the blinding** of the spendable side directly, without the per-transfer ECDH channel the upstream sketch leans on. We verified both recoveries, and their negatives, against the unmodified circuit library (`experiments/clawback-poc`, 5/5 tests).

11.3 · Proposal: escrow the viewing key at registration

The register circuit is five constraints (R1–R5) and **already computes `vk` in-circuit** (R2). The account also already binds an `auditor_id` at registration. We propose adding, at that point, an auditor-bound ciphertext of `vk`:

#	ADDED CONSTRAINT
R6	`R_esc = r_esc · H`, `r_esc ≠ 0` — ephemeral key for the escrow
R7	`s_esc = ECDH(r_esc, K_aud)` — shared secret with the account's chosen auditor
R8	`vk_masked = vk + Poseidon2(ESC_VK, s_esc.x, addr_f)` — masked viewing key, emitted

Four to six constraints over gadgets the circuit family already ships (`scalar_mul`, `ecdh`, `poseidon_with_domain`) — the same pattern the transfer circuits use for their auditor ciphertexts. Idiomatic, not new machinery.

11.4 · Why this is safe

It grants no new knowledge. The auditor already recovers `v_s` from the checkpoint and every transfer amount from the auditor channel; that is its designed role. The escrow converts knowledge the auditor already holds into knowledge it can *prove in circuit*. Marginal privacy loss: nil.

It grants no spending power. Moving value requires `sk` — the transfer circuit's T1 binds `Y = sk · H` — and `vk = Poseidon2(VIEWING_KEY, sk, addr_f)` is one-way. An auditor holding `vk` can open commitments; it cannot produce a transfer.

It grants no power to forge attestations. The predicate circuit of §4.2 takes `sk` as a private witness and derives `vk` from it (D1). A party holding `vk` alone cannot satisfy D1, so it cannot manufacture a `disclose_balance_ge` proof in a holder's name.

It is scoped by the account. The escrow is encrypted to the auditor key the account itself selected at registration; a deployment restricting that choice (upstream's `ApprovedAuditorHooks` pattern) restricts the escrow with it.

11.5 · What it unlocks: partial seizure, not merely total

Total seizure would not need any of this. Zeroing both commitments to the identity point leaves an opening of `(0, 0)` that everyone knows, so the holder still operates afterwards; the only hard part is establishing the amount.

Partial seizure is what needs the escrow, and gets it for free. With `vk` the auditor derives not only the current blinding but the *post-seizure* one,

```
r_s_new = derive_spend_r(vk_A, sigma_new)
```

which is the protocol's own canonical rule — the same value the wallet would have derived. The rewritten balance therefore stays openable by the holder through ordinary checkpoint recovery, with no out-of-band handoff. The seize circuit then reduces to opening proofs with the seized amount `alpha` as a **public** input and a range constraint `alpha ≤ v_s + v_r`.

This round-trip is one of the verified tests: the auditor writes the post-seizure state under the canonical rule, and the holder — using only `sk` and the event, as in ordinary wallet recovery — re-derives the same blinding and opens the same commitment.

The circuit exists (`circuits/seize`, constraints Z1–Z7). Z1 binds the escrowed `vk` to the target's on-chain viewing key, so a proof built for one account cannot be applied to another; Z6/Z7 force the post-seizure commitment and checkpoint to match `remaining` under the canonical derivations, so the authority can neither short-change the holder nor desynchronize its wallet. At 93 ACIR opcodes it is cheaper than the transfer circuit itself.

It also verifies on-chain. `contracts/velum-seize` reads `PVK_A`, `C_spend` and `C_receive` from the token, assembles the eleven public inputs and verifies the proof — so a seizure is settled against state the authority cannot substitute. Two limits are worth stating in the same breath: no value moves, because rewriting the commitments needs a `seize` entry point inside the token that we did not fork; and the escrow below is simulated with a test account's keys. What is established is that the verification half works on-chain, which was the half in doubt.

The *receiving* side closes as well. `C_receive` accumulates two kinds of contribution: confidential transfers, whose per-transfer openings the auditor already recovers from its designed channel, and deposits — which the contract adds as `amount * G` with **zero blinding**, the amount being public. Every term of the receiving commitment therefore has an opening the auditor can produce.

11.6 · Cost

The cryptography is the cheap part; the migration is not.

- A changed register circuit means a **new verification key and no backward compatibility**: every existing account must re-register.
- It forks the token contract's register entry point and public-input assembly.
- It requires auditor-side SDK work: event replay, derivation, proving.

For a protocol still in developer preview, before mainnet and before large-scale registration, this is the cheapest moment such a change will ever have.

11.7 · Residual questions

Auditor key rotation. Escrow ciphertexts bind to the auditor key current at registration. Rotation needs either retention of the old key for historical accounts or a re-escrow flow.

Governance of a write surface. Seizure lets an admin-and-auditor pair move value without the holder. Deployments should separate freeze, policy and seize roles, and consider a timelock and a mandatory event trail. The two-party split is a property of the construction, not a cryptographic impossibility.

Regulatory sufficiency. Freeze is immediate and unilateral; seizure is coordinated and slow. In most regimes the legal act happens off-chain and the on-chain move is execution — so freeze is what must never fail, and seizure is the convenience.

12 Conclusion

Confidentiality built on commitments does not merely make quantitative compliance harder; it moves where the check can happen. The party that can prove a statement about a balance is the party that holds it. Once that is accepted, the design follows: the holder proves, a contract verifies, and the result is a fact on the ledger that other contracts can consume — while the value stays hidden from everyone, verifier included.

Velum implements that for the two rules a regulated fund needs first, on top of an unmodified upstream token, using a circuit its own specification already describes. The remaining hard problem — predicates over encrypted aggregates across accounts — is, as far as we can determine, unsolved on any chain.

Appendix A — The gap in upstream's own words

Every load-bearing claim in §§2, 3 and 11 is a statement about someone else's codebase, so this appendix sources each one to a line of that codebase. Nothing here is our characterisation: the quotes are verbatim and the signatures are copied as they appear.

How to verify. Two revisions are involved, and the distinction matters. The Confidential Token *implementation* lives only on `feat/confidential-verifier-ultrahonk`, which we pin at `539968f`; the

specifications were merged to `main`, which we read at `9b5ed96` (2026-07-31). Paths below are relative to `packages/tokens/src/` in `OpenZeppelin/stellar-contracts`.

A.1 · The authorization interface cannot see value

`confidential/compliance/mod.rs:41`

```
pub trait Policy {
    /// Returns `true` iff `account` is authorized to interact with
    /// `token`.
    fn is_authorized(e: Env, account: Address, token: Address) -> bool;
}
```

An address, a token, a boolean. The specification states the consequence plainly (`confidential/docs/COMPLIANCE.md:81`):

> "Membership management, list semantics, and identity proofs live entirely inside the policy contract. **The token's only agreement with the policy is the boolean return value.**"

The lifecycle hooks draw the same boundary from the other side. `confidential/mod.rs:175` and `:185`:

```
fn on_deposit(e: &Env, from: &Address, to: &Address, amount: i128) {}
fn on_transfer(e: &Env, from: &Address, to: &Address, payload: &TransferPayload) {}
```

A deposit carries a cleartext `i128`. A confidential transfer carries a payload of commitments and a proof — there is no amount to pass, and none is passed. This is §3's argument, in two signatures.

A.2 · The two compliance worlds have incompatible signatures

The RWA module's compliance hooks, in the same library, expect exactly what the confidential side cannot provide. `rwa/compliance/modules/mod.rs:166`:

```
fn on_transfer(
    e: &Env,
    from: AccountSnapshot,
    to: AccountSnapshot,
    amount: i128,
    kind: TransferKind,
    token: Address,
);
```

and `rwa/compliance/mod.rs:92`:

```
pub struct AccountSnapshot {
    pub address: Address,
    /// The wallet's total token balance, before the operation.
    pub balance: i128,
    /// The partially-frozen portion of `balance`, before the operation.
    pub frozen: i128,
}
```

Cleartext amount, cleartext balance, cleartext frozen portion. Every shipped quantitative module — `compliance-max-balance`, `compliance-supply-limit`, `compliance-initial-lockup-period`, `compliance-time-transfers-limits` — is written against this. None of them can be wired to a confidential token, and the reason is a type signature, not an oversight.

A.3 · Upstream says the identity bridge is missing

Issue **#766, "RWA: Confidential support"** — opened 2026-07-06, still **open** at the time of writing, milestone *Release Candidate v0.9.0*:

> "Adds confidential-balance support to the RWA token, integrating the confidential token module's private balances and transfers with RWA's compliance and identity-verification extensions, so regulated tokens can hide transfer amounts while retaining on-chain compliance enforcement."

That is the shape of `velum-policy`, written by the maintainers as future work. We also checked the released artifacts: the newest tag at the time of writing (`v0.8.0-rc.3`) contains no cross-reference between the `confidential` and `rwa` modules in either direction.

A.4 · The clawback problem, stated and deferred by upstream

`confidential/docs/COMPLIANCE.md:185` — why an issuer's ordinary clawback cannot reach one holder:

> "Once an account deposits into the contract, the underlying SEP-41 ledger lists the token contract as the holder of those funds, not the depositor. An issuer's SAC-level `clawback(token_address, amount)` call would drain the pool, debiting unrelated accounts."

And `COMPLIANCE.md:218` — the sentence that defines §11 of this paper. It names both the obstacle and, in its last clause, the shape of the remedy *(mathematical notation simplified from the LaTeX source; wording otherwise verbatim)*:

> "…where `r_s` is recovered via the same path the wallet uses for checkpoint recovery (`DESIGN.md` §5.2): `r_s = Poseidon(δ_spend_r, vk_A, σ_old)`. **Because the clawback circuit does not have access to `vk_A`**, the spendable-balance side of the proof binds via the consistency of `b̃_aud,s_old` with `C_spend` at the time of the last owner-initiated proof. **The follow-up revision will pin down whether `r_s` is supplied as a private witness with an auxiliary opening proof or derived in-circuit from a separately escrowed value.**"

Three things are established by that passage alone. The blinding is a deterministic function of `vk_A` — their equation, not ours. The obstacle is that the circuit lacks `vk_A`. And one of the two candidate remedies they list is *a separately escrowed value*. §11 takes that second option and says what the escrowed value should be: `vk_A` itself.

A.5 · The predicate circuit is specified, and unimplemented

`confidential/docs/SELECTIVE_DISCLOSURE.md:374` specifies the circuit implemented in `circuits/disclose_balance_ge`:

> "Two `circuit_id` shapes are exposed: a **predicate-only** form (`disclose_balance_ge` / `disclose_balance_le`) that includes DB4 and omits U1–U3, where the proof's mere validity asserts the predicate; and a **value-revealing** form (`disclose_balance_value`)..."

The §15.1 circuit inventory lists four disclosure circuits: `disclose_recipient`, `disclose_sender`, `disclose_auditor`, `disclose_balance`. The reference implementation distributes two. The provenance check for the other two is recorded at the end of this paper.

A.6 · On-chain verification is named, then excluded

`SELECTIVE_DISCLOSURE.md:542`:

> "These circuits do *not* register with the on-chain verifier set (DESIGN_cont.md §10). **They are verified entirely off-chain.**"

The exclusion is deliberate, and §14 records what the excluded thing would look like — including that the predicate shape is the one it would take:

> "...can be added by trivially dropping U1–U3 and exposing `v_transfer` as a public input. **This is also the proof shape an on-chain verifier would consume (§5.4).**"

The enum that would have to carry such a circuit confirms the exclusion in code (`confidential/verifier/mod.rs:100`):

```
Register = 0,
Withdraw = 1,
Transfer = 2,
SpenderTransfer = 3,
SetSpender = 4,
RevokeSpender = 5,
```

Six transaction circuits, no disclosure variant. `velum-attest` therefore runs its own single-entry registry rather than extending a closed enum.

A.7 · What this appendix is not

It is not a claim that the maintainers were unaware, careless, or slow. The opposite: each gap above is documented by them, in their own repository, in the same commits that ship the working parts. A specification that names its own open questions is a good specification. Velum's contribution is to answer three of those questions with running code — and, where an answer is still a proposal (§11's migration), to say so.

Reproducibility

```
noirup --version 1.0.0-beta.11 && bbup -v 0.87.0
cd circuits/disclose_balance_ge && nargo test
nargo execute witness
bb write_vk --scheme ultra_honk -b target/disclose_balance_ge.json -o target/
bb prove    --scheme ultra_honk -b target/disclose_balance_ge.json -w target/witness.gz -o target/
bb verify   --scheme ultra_honk -k target/vk -p target/proof -i target/public_inputs
cd ../../contracts && stellar contract build
```

Toolchain: `stellar-cli ≥ 25.2`, Rust with `wasm32v1-none`, soroban-sdk `=26.1.0`, OpenZeppelin `stellar-contracts` rev `539968f`. Noir rejects non-ASCII source, comments included.

Provenance of the novelty claim

`disclose_balance`, `disclose_balance_ge`, `disclose_balance_le` and `disclose_auditor` appear only in upstream's specification, never in code. Verified 2026-08-04/05 across: full-text search of three clones; all 16 branches and 10 tags; the complete git history (no such file in any commit); commit messages; issues and pull requests (zero matches); and authenticated code search across GitHub. On-chain disclosure verification likewise has no public implementation — the specification states the circuits do not register with the on-chain verifier set, and the `CircuitType` enum carries no disclosure variant.

We claim only what that supports: **no public implementation exists in any repository, branch, tag or history.** Private work at OpenZeppelin cannot be excluded.

Status. Developer-preview software on unaudited primitives. Testnet only, no real value. Not an offer or a solicitation; the regulatory mapping is engineering intent, not legal advice.